

Web Privacy TP1 — Stateful Tracking

Lab Report

Nour El Houda Eliamani

September 27, 2026

1 Manipulating Cookies

Goal: recognize Alice when she comes back to the website, using a cookie that stores a unique identifier (`aid`).

1.1 First visit

On the first visit, the browser has no cookie. The server creates a random identifier and sends it in the response.

Welcome to Alice's Web Journey

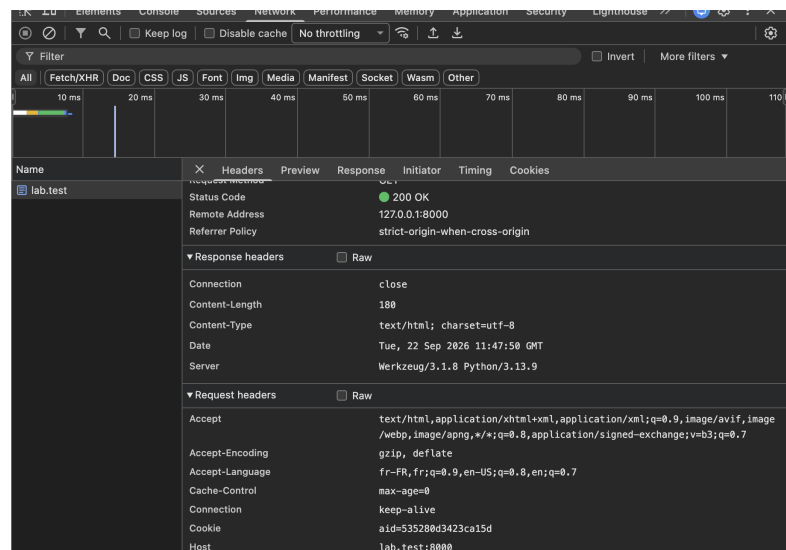


Figure 1: First visit: the server sends `Set-Cookie: aid=535280d3423ca15d; Path=/'`.

Observation: the response contains a `Set-Cookie` header, and the browser saves the cookie.

1.2 Return visit after restarting the browser

We closed the browser, opened it again and went back to `http://lab.test:8000`.

Welcome to Alice's Web Journey

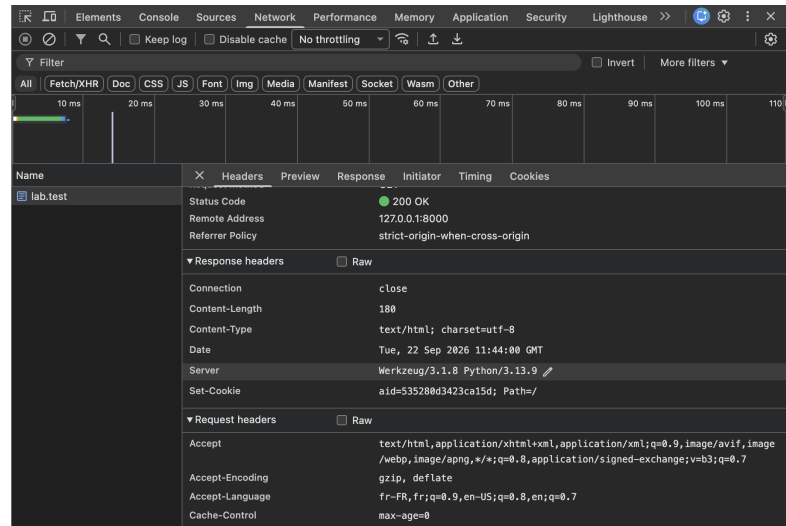


Figure 2: Return visit: the browser sends Cookie: aid=535280d3423ca15d in the request.

Observation: the server does not send Set-Cookie again. The browser sends the cookie back automatically in the Cookie request header.

1.3 Server-side view

```
--- REQUEST ---
Cookie header: None
aid = 535280d3423ca15d | new visitor? True
127.0.0.1 - - [22/Sep/2026 11:44:00] "GET / HTTP/1.1" 200 -

--- REQUEST ---
Cookie header: aid=535280d3423ca15d
aid = 535280d3423ca15d | new visitor? False
127.0.0.1 - - [22/Sep/2026 11:47:50] "GET / HTTP/1.1" 200 -
```

Figure 3: Flask terminal: new visitor on the first request, returning visitor on the second.

Observation: on the first request the cookie header is `None`, so the server sees a new visitor. On the second request it receives the same `aid`, so it recognizes the browser.

Q: Is the cookie still present? Does it have the same identifier?

Yes. After restarting the browser, the cookie was still there with the same identifier (535280d3423ca15d). The server recognized the same browser, even though HTTP is stateless and Alice never logged in.

Note: the cookie has no **Expires** or **Max-Age**, so it is a *session cookie* and should normally be deleted when the browser closes. It probably survived because the browser restores the previous session (“Continue where you left off”).

2 Challenge 1 — Third-Party Tracking

Setup: two publishers and one tracker, each running as a separate Flask app:

- publisher-one.test:8001 — *Tech Daily* (technology news);

- `publisher-two.test` — *Wander* (travel stories);
- `tracker-one.test:9000` — the tracker, embedded in both publishers with an `iframe`:
`/track?publisher=<name>&page=<page>`.

The tracker sets a persistent cookie `tid` (`Max-Age=31536000`, one year) and logs every request with the identifier, the publisher and the page.

2.1 Attempt 1 — HTTP

Tech Daily — technology news

home

[Home](#) | [AI chips](#) | [Quantum computing](#)

Article content for home.

tracker-one sees you as b47b14585f5f3913

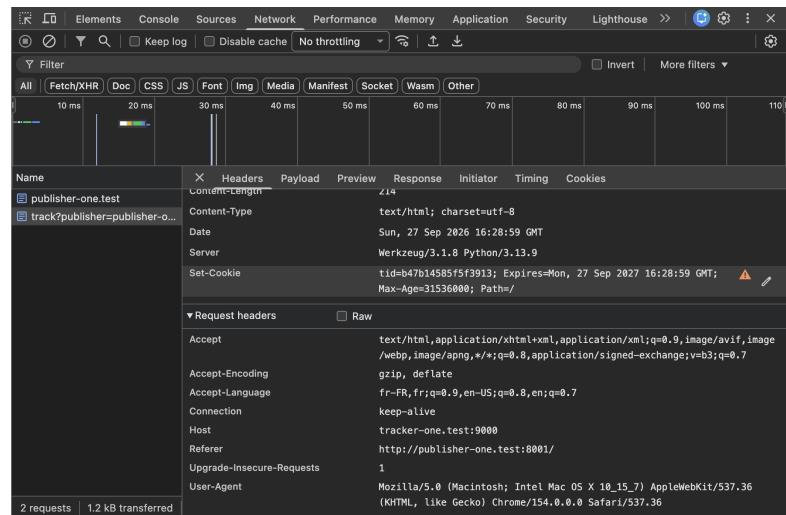


Figure 4: Publisher one: the `iframe` request goes to `tracker-one.test:9000` with `Referer: publisher-one.test:8001`. The tracker sets `tid=b47b14585f5f3913`. Chrome shows a warning icon next to `Set-Cookie`.

Observation: the tracker learns which publisher Alice is on from the query string and the `Referer` header. It tries to store `tid`, but Chrome shows a warning on the cookie.

Wander — travel stories

home

[Home](#) | [Kyoto in autumn](#) | [Hiking Patagonia](#)

Travel article content for home.

tracker-one sees you as e1c4fb2f4bc5eed7

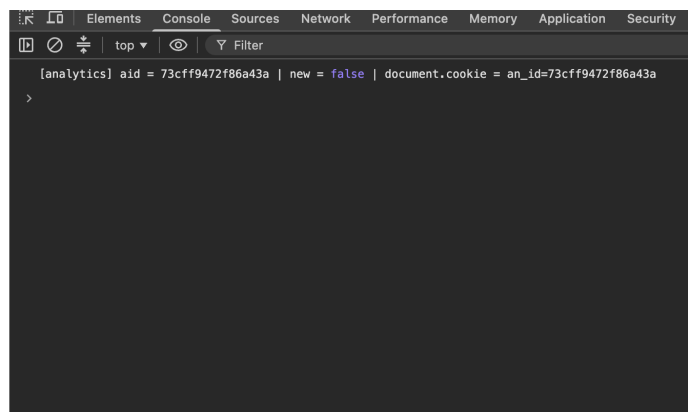


Figure 5: Publisher two: the same tracker now sees Alice as `e1c4fb2f4bc5eed7`, a different identifier.

Observation: on the second publisher, the tracker gives Alice a **new** identifier. So it cannot link the two visits.

Why are the identifiers different?

In the iframe, `tracker-one.test` is a *third party* (it is not the site in the address bar). The cookie has no `SameSite` attribute, so Chrome uses `SameSite=Lax` by default. A Lax cookie is not stored or sent in a cross-site iframe. The browser therefore never sends `tid` back, and the tracker creates a new identifier on every visit. To work in a third-party context, the cookie needs `SameSite=None; Secure`, and `Secure` requires HTTPS.

2.2 Attempt 2 — HTTPS with `SameSite=None; Secure`

We switched the sites to HTTPS with a self-signed certificate.

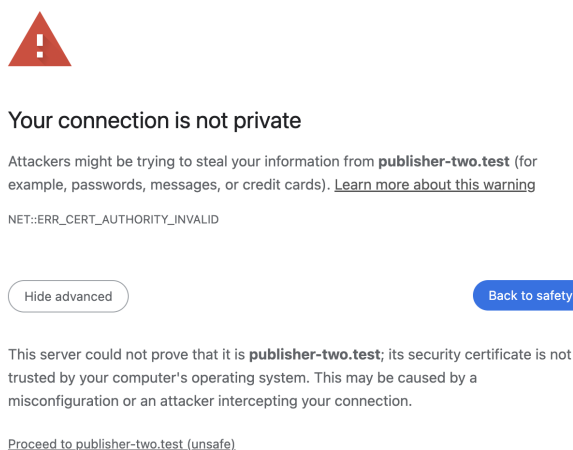


Figure 6: Chrome rejects the self-signed certificate (`NET::ERR_CERT_AUTHORITY_INVALID`).

Tech Daily — technology news

home

[Home](#) | [AI chips](#) | [Quantum computing](#)

Article content for home.



Wander — travel stories

home

[Home](#) | [Kyoto in autumn](#) | [Hiking Patagonia](#)

Travel article content for home.

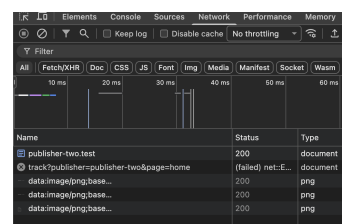


Figure 7: The tracker iframe does not load on either publisher. In the Network panel, `track?publisher=publisher-two&page=home` fails.

Observation: we can accept the warning for the publisher page, but not for the tracker, because the warning page cannot be shown inside an iframe. The request to the tracker fails, so no cookie is set.

Why does the iframe fail?

The browser does not trust the tracker's certificate, and it blocks the request silently inside the iframe. The fix is to open `https://tracker-one.test:9000` directly once and accept the warning, or to create trusted local certificates (for example with `mkcert`).

2.3 Conclusion

A third-party tracker can only follow Alice across sites if the browser stores and sends its cookie in the cross-site context. Modern browsers block this by default (`SameSite=Lax`), so the tracker needs HTTPS and `SameSite=None; Secure`. When the cookie is blocked, each publisher visit looks like a new user, and the tracker cannot build one profile.

3 Challenge 2 — First-Party Analytics

Setup: an analytics server runs on `analytics.test:9100`. Both publishers include its script:

```
<script src="http://analytics.test:9100/static/analytics.js"></script>
```

The script checks for an `an_id` cookie, creates a random ID if it does not exist, and saves it with `document.cookie`. It then sends the page view to `analytics.test/collect`.

3.1 Cookies stored for each publisher

Tech Daily — technology news

home

[Home](#) | [AI chips](#) | [Quantum computing](#)

Article content for home.

tracker-one sees you as 291465dccab65e75

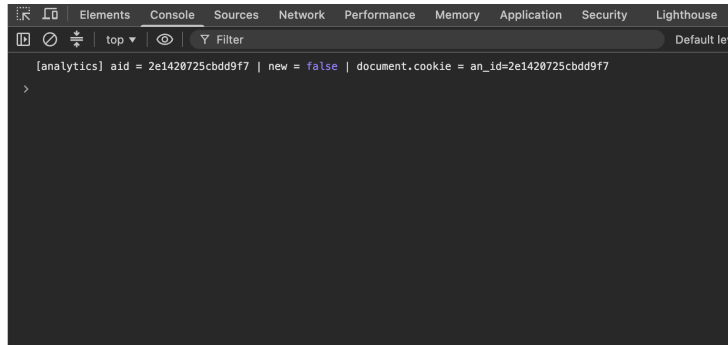


Figure 8: Publisher one: the script logs `an_id=2e1420725cbdd9f7`.

Tech Daily — technology news

home

[Home](#) | [AI chips](#) | [Quantum computing](#)

Article content for home.

tracker-one sees you as 291465dccab65e75

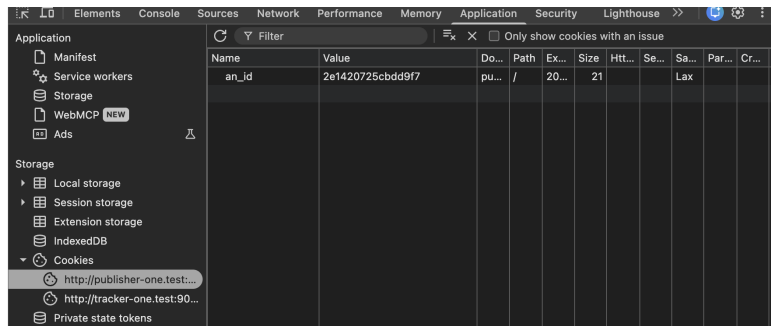


Figure 9: Application panel: `an_id` is stored under `publisher-one.test`, not under `analytics.test`. It has `SameSite=Lax` and no `HttpOnly` flag.

Wander — travel stories

home

[Home](#) | [Kyoto in autumn](#) | [Hiking Patagonia](#)

Travel article content for home.

tracker-one sees you as e1c4fb2f4bc5eed7

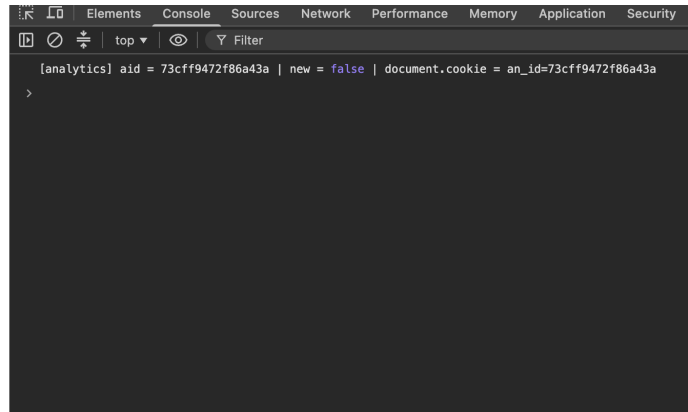


Figure 10: Publisher two: the same script logs a different value, `an_id=73cff9472f86a43a`.

Publisher	Cookie value	Cookie domain
Tech Daily	<code>an_id=2e1420725cbdd9f7</code>	<code>publisher-one.test</code>
Wander	<code>an_id=73cff9472f86a43a</code>	<code>publisher-two.test</code>

Q: Are the two publishers assigned the same identifier or different identifiers?

Different identifiers.

Q: Why does this behaviour occur?

The script runs inside the publisher page, so `document.cookie` creates a *first-party* cookie for the publisher's domain. Cookies are isolated by domain: the script on Wander cannot read the cookie stored for Tech Daily. So it does not find a cookie and creates a new ID.

3.2 Requests received by the analytics server

Tech Daily — technology news

home

[Home](#) | [AI chips](#) | [Quantum computing](#)

Article content for home.

tracker-one sees you as 5ecbba95b4ebc450

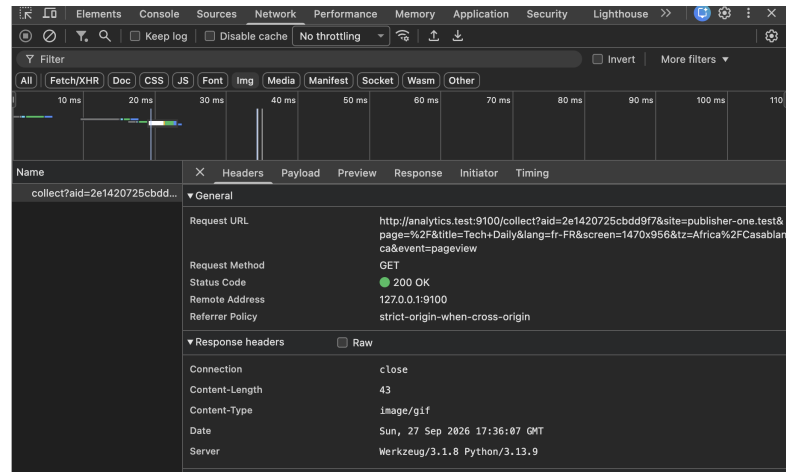


Figure 11: The analytics request: the ID and the browsing data are sent in the URL of `/collect`.

Observation: the request to `analytics.test:9100/collect` contains `aid`, `site`, `page`, `title`, `lang`, `screen`, `tz` and `event` in the query string. The server answers with a small `image/gif`.

Q: Does the analytics server automatically receive the first-party cookies created in the publishers' contexts?

No. The browser only sends a cookie to the domain that owns it. `an_id` belongs to `publisher-one.test`, so it is not attached to requests to `analytics.test`. This is why the script has to copy the ID into the URL (`aid=...`).

3.3 Profiles reconstructed by the analytics service

Profiles held by analytics.test

5 events, 2 distinct identifiers

aid = 2e1420725cbdd9f7

seen on: **publisher-one.test**

1. 17:32:30 — publisher-one.test/ [pageview]
2. 17:33:06 — publisher-one.test/ [pageview]
3. 17:36:07 — publisher-one.test/ [pageview]

aid = 73cff9472f86a43a

seen on: **publisher-two.test**

1. 17:37:15 — publisher-two.test/ [pageview]
2. 17:41:25 — publisher-two.test/ [pageview]

Figure 12: The analytics server: 5 events, 2 distinct identifiers.

Q: One combined profile or multiple separate profiles?

Two separate profiles. `2e1420725cbdd9f7` only contains visits to publisher one, and `73cff9472f86a43a` only contains visits to publisher two. Alice is the same person, but because each publisher has its own first-party cookie, the analytics service sees two different users. To link them, it would need extra information, for example a login, a fingerprint (language, screen size, time zone are already collected) or cookie syncing.